

EVOAGENT: AN EVOLVABLE AGENT FRAMEWORK WITH SKILL LEARNING AND MULTI-AGENT DELEGATION

Aimin Zhang^{1*}

¹Focus AI Center, Focus Technology Co., Ltd.
zhangaimin@focuschina.com

Fuwei Jia¹

¹Focus AI Center, Focus Technology Co., Ltd.
jiaafuwei0921@focuschina.com

Jiajing Guo¹

¹Focus AI Center, Focus Technology Co., Ltd.
guojiajing@focuschina.com

Chen Lv¹

¹Focus AI Center, Focus Technology Co., Ltd.
lvchen1018@focuschina.com

Boyu Wang²

²Engineering Research Center of Digital Forensics, Ministry of Education,
Nanjing University of Information Science and Technology
wangboyu@focuschina.com

Fangzheng Li^{1,3}

¹Focus AI Center, Focus Technology Co., Ltd.

³Nanjing University of Science and Technology
lifangzheng@focuschina.com

ABSTRACT

This paper proposes EvoAgent—an evolvable large language model (LLM) agent framework that integrates structured skill learning with a hierarchical sub-agent delegation mechanism. EvoAgent models skills as multi-file structured capability units equipped with triggering mechanisms and evolutionary metadata, and enables continuous skill generation and optimization through a user-feedback-driven closed-loop process. In addition, by incorporating a three-stage skill matching strategy and a three-layer memory architecture, the framework supports dynamic task decomposition for complex problems and long-term capability accumulation. Experimental results based on real-world foreign trade scenarios demonstrate that, after integrating EvoAgent, GPT5.2 achieves significant improvements in professionalism, accuracy, and practical utility. Under a five-dimensional LLM-as-Judge evaluation protocol, the overall average score increases by approximately 28%. Further model transfer experiments indicate that the performance of an agent system depends not only on the intrinsic capabilities of the underlying model, but also on the degree of synergy between the model and the agent architecture. Code, data, and documents will be released at <https://github.com/Focus-AI-Center/Mentarc-EvoAgent.git>.

Keywords Evolvable Agent · Skill Self-Learning Mechanism · Harness Engineering

1 Introduction

Large Language Model (LLM) agents have gradually emerged as a core paradigm for achieving autonomous task execution in complex and open environments. By integrating capabilities such as reasoning, planning, tool invocation, and environment interaction, LLM agents are able to accomplish end-to-end task solving within a unified framework [1, 2, 3]. In professional domains such as software engineering, scientific computing, financial analysis, and enterprise workflow orchestration, the tasks faced by agents increasingly exhibit multi-step, cross-domain, and highly structured characteristics. Their complexity has significantly exceeded the capability boundaries supported by isolated invocations of atomic tools. To bridge this gap, researchers have progressively introduced the concept of “Skill” as a higher-level encapsulation of capability [4, 5]. Unlike atomic tools that provide a single function, skills are defined as structured

*Corresponding author: zhangaimin@focuschina.com

multi-file collections, typically including workflow instructions, executable scripts, domain knowledge references, and reusable logic modules, thereby offering systematic and reusable process support for complex task execution [6].

Although the skill mechanism demonstrates significant value in enhancing complex task execution, existing paradigms for skill acquisition and application still suffer from several key limitations. First, mainstream approaches heavily rely on manual authoring, resulting in high labor costs, limited scalability, and difficulty in maintaining consistent quality across domains. Second, manually constructed skills often exhibit human–AI cognitive misalignment: workflows optimized based on human intuition may not align with the intrinsic reasoning patterns of large language models, leading to performance degradation in highly structured domains such as natural sciences [4, 7]. Third, most existing self-evolution methods primarily focus on heuristic optimization at the level of individual tools or prompts, and do not yet support systematic iterative generation, optimization, and validation of complex multi-file skill packages. Finally, many self-improvement frameworks rely on real annotated supervision [4] or intensive expert feedback, conditions that are often difficult to satisfy in real-world deployment scenarios.

Meanwhile, advances in multi-agent systems indicate that task decomposition and role specialization provide significant advantages in solving complex problems [8, 9]. Within hierarchical orchestration architectures, systems support configurable sub-agents and explicit task delegation: the main agent decomposes high-level objectives and assigns subtasks to specialized sub-agents with domain-specific capabilities. This design effectively mitigates context window limitations, reduces cognitive load, and improves parallel processing capacity and system robustness. However, mainstream multi-agent frameworks typically adopt static role definitions and fixed task routing strategies, lacking adaptive mechanisms that dynamically evolve delegation logic, skill sets, and collaboration patterns based on historical experience. Moreover, the integration between skill self-evolution mechanisms and hierarchical multi-agent task delegation remains limited, resulting in a structural disconnect between individual skill learning and global task scheduling optimization. To address these challenges, this paper proposes EvoAgent—an evolvable large language model agent framework that integrates autonomous skill learning with hierarchical multi-agent task delegation mechanisms.

From a design philosophy perspective, EvoAgent follows the systems engineering paradigm of Harness Engineering. The concept of Harness Engineering was systematically introduced by Mitchell Hashimoto in 2026, emphasizing that as large models rapidly increase in capability, engineering focus should shift from “enhancing model capability” to “harnessing model capability”—that is, constraining, guiding, and integrating model behavior through a structured external control layer (Harness) [10]. OpenAI further articulated the three pillars of Harness—Constraints, Observability, and Feedback Loops—in the report **Harness Engineering: Leveraging Codex**, highlighting that the reliability of agent systems originates from the engineered shell built around the model rather than from the model’s intrinsic reasoning ability [11].

At the architectural level, Birgitta Böckeler and Martin Fowler proposed a 2×2 structural matrix of Guide/Sensor × Computational/Inferential to characterize the structural roles of Harness in reasoning guidance and system monitoring [12]. Furthermore, Paul Iusztin formalized Agentic Harness Engineering as a systematic framework consisting of eight core components, including task orchestration, state management, execution monitoring, and feedback loops, and introduced the engineering perspective of “LLMs as the New Operating System” [13].

Different from skill-based agents that focus solely on automated co-evolution validation mechanisms, EvoAgent concretely implements the Harness engineering philosophy as a user-centered dual-loop system encompassing both online execution and post-session evolution. The online loop constructs a deterministic execution trajectory through enforced skill matching, context assembly, and task delegation pipelines; the offline loop enables capability evolution via asynchronous session review and skill extraction mechanisms. Supported by this Harness framework, EvoAgent achieves controllable, evolvable, and engineering-stable capability growth in real-world foreign trade business environments.

Specifically, the framework includes the following four core innovations:

- Proposes a structured skill representation method that organizes reusable procedural knowledge into multi-file skill packages equipped with lazy-loading reference mechanisms and evolutionary metadata, supporting persistent storage and dynamic on-demand invocation.
- Establishes a user-centered skill self-evolution mechanism that drives iterative optimization by tracking skill usage frequency and execution success rate, alleviating human–AI cognitive misalignment while removing dependence on real labeled data.
- Designs a hierarchical sub-agent delegation architecture that enables the main agent to create specialized sub-agents with independent context spaces on demand, thereby improving execution efficiency for complex multi-stage tasks.

- Constructs a three-layer memory system (SOUL.md/USER.md/MEMORY.md) combined with a dialogue history compression mechanism to achieve long-term context retention and cross-session knowledge accumulation.

In summary, EvoAgent targets complex real-world foreign trade application scenarios and constructs a unified agent framework that integrates adaptability and evolvability, enabling agents to continuously enhance their capabilities through skill learning and collaborative task execution. This study provides a systematic foundational framework for the next generation of autonomous agents, advancing scalability, adaptability, and practical deployability, and demonstrating strong potential for real-world deployment in enterprise environments and professional domains.

2 Related Work

2.1 Skill Learning and Self-Evolution in LLM Agents

Large language models have made rapid progress in tool invocation and complex task execution. Among these advances, “Skill” has emerged as a higher-level capability encapsulation distinct from atomic tools, designed to support multi-step, cross-domain professional task solving. Anthropic formally introduced the concept of agent skills [5, 14], defining them as structured multi-file packages composed of workflow instructions, executable scripts, and domain reference files, thereby significantly enhancing task completion capabilities in professional scenarios. In this context, constructing agent systems with autonomous planning and tool invocation abilities has become a key research focus. Furthermore, some studies have begun to address the problem of “capability growth,” namely how agents can accumulate experience during continuous task execution and transform it into reusable and transferable capability units. Existing research typically follows several paths [4]:

- Strategy summarization based on execution trajectories, abstracting decomposition steps of successful tasks into templated operational workflows;
- Reflection-based mechanisms that analyze failure trajectories and revise strategies accordingly;
- Retrieval-augmented memory approaches that invoke historical successful cases in new tasks to improve reasoning quality [15, 16].

Although these methods have achieved certain improvements in enhancing single-agent stability and task success rates, their core capability structures still rely on fixed prompting frameworks or predefined strategy patterns. In existing public research, relevant distinctions have not yet been systematically modeled, and there remains a lack of comprehensive modeling of the skill lifecycle (including generation, evaluation, consolidation, and elimination), as well as targeted designs of self-evolving agents for specific scenarios [17]. Moreover, current work mainly focuses on internal strategy optimization within a single agent, with limited exploration of how skill learning mechanisms can co-evolve with system-level architectural expansion.

Therefore, research on evolvable agents still faces two key challenges: first, how to abstract task execution trajectories into composable and transferable structured skill units; second, how to integrate skill learning mechanisms with architectural evolution so that capability expansion is reflected not only at the strategic level but also at the organizational and collaborative levels.

Against this backdrop, EvoAgent treats skills as continuously accumulable structured capability assets, supporting their generation, evaluation, and reuse through a closed-loop feedback mechanism, thereby enabling sustained capability growth oriented toward long-term task distributions.

2.2 Multi-Agent Collaboration and Structured Task Architectures

In complex task scenarios, large language models often face limitations in context window size and excessive cognitive load. To alleviate these bottlenecks, researchers commonly adopt task decomposition and structured execution mechanisms, breaking down overall objectives into subtasks and enhancing stability and controllability through staged reasoning. A representative method is the ReAct framework, which alternates reasoning and tool invocation to form a “Think–Act–Observe” closed-loop process, thereby strengthening complex task handling capabilities [1].

Building upon this, some studies introduce multi-agent collaboration mechanisms, leveraging role specialization and dialog interaction to achieve task decomposition and cooperative problem-solving. For example, CAMEL [18] simulates collaborative task-solving by assigning different roles to agents engaged in dialogue, while AutoGen [19] proposes an orchestratable multi-agent conversation framework that supports the separation of planning and execution for complex

tasks. These approaches typically employ predefined roles and fixed communication protocols to complete tasks through multi-round interactions.

However, most existing multi-agent frameworks focus on collaborative reasoning at the level of single tasks. Their collaboration structures are usually statically determined during system design and lack the capability to continuously optimize based on historical interaction outcomes. At the same time, role specialization is often primarily organizational in nature and is not deeply integrated with transferable skill accumulation mechanisms, making it difficult to support long-term capability accumulation and system self-evolution.

On the other hand, some works have begun to explore self-improvement and experiential reflection mechanisms in agents. Reflexion [20] introduces a language feedback loop to revise strategies; Voyager [21] constructs a skill library to support continuous capability expansion in open environments; Generative Agents [22] investigate long-term memory and behavioral evolution mechanisms in simulated settings. While these methods emphasize experience accumulation and capability enhancement, their capability updates often remain implicit and lack structured, transferable skill representations.

Overall, existing research has made important progress in task structuring, multi-role collaboration, and reflective capability improvement. However, in real-world business scenarios, there remains a lack of a unified framework that simultaneously integrates structured process control, lightweight multi-role collaboration, and explicit skill accumulation mechanisms. Particularly in industrial environments that emphasize deployment cost and system scalability, achieving sustainable evolution and capability transfer without over-reliance on complex multi-agent orchestration remains an important research challenge.

2.3 Harness Engineering

As the capabilities of large language models rapidly improve, research emphasis has gradually shifted from “how to enhance model reasoning ability” to “how to systematically harness model capability.” Harness Engineering, an emerging engineering paradigm in recent years, emphasizes constraining, guiding, and integrating model behavior through an external structured control layer, thereby constructing stable, observable, and evolvable agent systems.

Mitchell Hashimoto systematically introduced the concept of Harness Engineering, arguing that the model itself is merely the “reasoning kernel” within a system, while the true determinant of system reliability lies in the engineered shell (Harness) built around it [10]. This perspective extends the focus of agent development from prompt engineering to system-level execution orchestration and state management.

In the report **Harness Engineering: Leveraging Codex**, OpenAI further articulated three core pillars of Harness: Constraints, Observability, and Feedback Loops [11]. Constraints regulate the model’s output space; observability monitors reasoning and tool invocation trajectories; and feedback loops support continuous capability optimization.

From a system modeling perspective, Birgitta Böckeler and Martin Fowler proposed a Guide/Sensor $\times$ Computational/Inferential 2×2 analytical matrix [12], dividing Harness into reasoning guidance components and execution monitoring components, while distinguishing deterministic computational paths from generative inferential paths. This framework provides a unified structural lens for analyzing agent architectures.

Furthermore, Paul Iusztin proposed an eight-component system architecture within the Agentic Harness Engineering framework, including orchestration, execution, state management, tool interface, monitoring, feedback, strategy, and memory layers. He emphasized that large language models should be regarded as a new type of “operating system kernel,” with the Harness assuming the roles of scheduling and control [13].

Compared with traditional research on multi-agent collaboration or skill self-evolution, Harness Engineering places stronger emphasis on system-level controllability, observability, and evolvability. EvoAgent inherits and extends this paradigm by structurally separating the online execution loop from the offline evolution loop, achieving an engineering balance between strongly constrained execution and capability evolution.

3 Methodology

The EvoAgent framework is a user-centered evolvable agent architecture. Through continuous interaction with users, EvoAgent achieves autonomous iteration and optimization of skills. This section elaborates on the research methodology. We first present an overview of the research approach, then provide the theoretical modeling of EvoAgent, and finally introduce the system architecture. Figure 1 illustrates the overall architecture of EvoAgent. The EvoAgent architecture adopts a hierarchical design in which a main agent collaborates with sub-agents to accomplish task processing. The system consists of five layers: the API entry layer provides a unified interface; the orchestration layer handles intelligent

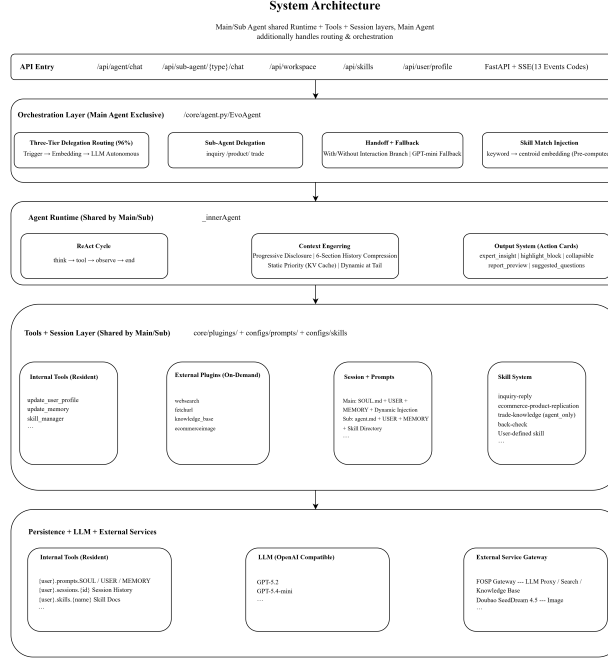


Figure 1: EvoAgent System Architecture

routing and task distribution; the runtime layer executes core logic; the tool and session layer provides execution resources; and the persistence layer manages data storage. The architectural highlights include a three-layer delegation routing mechanism and a shared runtime design, enabling efficient context management and tool invocation. The system employs the ReAct loop to implement reasoning and action, optimizes context utilization through a progressive disclosure strategy, and supports structured outputs and event-driven interaction.

3.1 Method Overview

The core objective of EvoAgent is to enable an agent operating in open foreign trade environments to autonomously acquire and continuously optimize skills without explicit manual programming or labeled data supervision. Inspired by the traditional *Human-in-the-loop* paradigm [23], its evolutionary mechanism can be summarized as a *User-in-the-loop* driven paradigm, where real user interaction feedback serves as the primary signal for capability evolution.

Overall, the EvoAgent workflow consists of the following three stages:

- **Context-aware matching:** Upon receiving user input, EvoAgent performs a three-stage retrieval process—keyword matching, vector embedding similarity matching, and LLM-based semantic matching—to select the skill unit most appropriate to the current task context from the skill repository.
- **Task execution and feedback collection:** The system injects the matched skill into the current context to complete task execution. During this process, it implicitly records statistical indicators such as skill invocation frequency and execution success rate, which serve as key feedback signals for subsequent skill evolution.
- **Session-based evolutionary update:** After the session concludes, EvoAgent initiates the evolutionary update process, updating user profiles and long-term memory by analyzing dialogue history, and extracting potential new skills or structurally optimizing existing skills based on accumulated usage data.

In Section 3.2, we formally model EvoAgent’s evolutionary mechanism and construct its corresponding Markov Decision Process (MDP) formulation; Section 3.3 further details the overall framework design and modular implementation of EvoAgent.

3.2 Problem Formulation

Task Environment Definition To formally model the self-evolution process of EvoAgent, we represent its task environment as a Markov Decision Process (MDP). We adopt an approximately observable state space to represent the

user interaction environment. The task environment of EvoAgent can thus be defined as $M_{EA} = (S, A, P, R)$, where S denotes the state space, composed of a quadruple including dialogue history, user profile, skill repository state, and context compression state; A represents the action space of the LLM, including skill selection, tool invocation, message generation, profile updates, and history compression; P denotes the state transition probability from the current state S to the next state S' given action a ; and the reward function $R(s, a, s')$ is modeled as a weighted combination of skill maturity, user profile update magnitude, and memory update magnitude.

State Space Definition The state space of EvoAgent is represented as a four-element tuple:

$$s = (h, u, S_{skills}, c).$$

The meanings of each component are summarized in Table 1:

Table 1: State Space Components

Component	Meaning	Corresponding Content
h	Dialogue history	Session records
u	User profile	USER.md + MEMORY.md
S_{skills}	Skill repository state	configs/skills/{name}/
c	Compression state	History compression level

Skill Space Definition A skill $skill \in S_{skills}$ is formally defined as:

$$skill = (name, desc, triggers, instr, refs, meta).$$

Here, name denotes the skill name; desc denotes the skill description; triggers is a list of keyword triggers used for matching; instr is the Markdown-formatted execution instruction; refs is a dictionary of reference files supporting lazy loading; and meta represents evolutionary metadata structured as $meta = (usage_count, success_rate, created_at, updated_at)$, corresponding to usage count, success rate, creation time, and last update time.

Skill Matching EvoAgent employs a three-stage skill matching strategy to balance efficiency and accuracy. The overall matching function is defined as:

$$M(u, S_{skills}) \rightarrow skill \cup \{\emptyset\} \quad (1)$$

Stage 1: Keyword Matching

In this stage, the system traverses all trigger lists of available skills, performing case-insensitive exact string matching, and returns the first matched result.

$$M_{keyword}(u, S_{skills}) = \{skill \in S_{skills} \mid \exists t \in skill.triggers, t.lower() \in u.lower()\} \quad (2)$$

Stage 2: Embedding Matching

Executed only if keyword matching fails, this stage encodes the user input using an external embedding API and computes cosine similarity with existing skill descriptions. The skill with the highest similarity exceeding a predefined threshold is selected. Here, $E(\cdot)$ denotes the embedding function and $\cos(\cdot)$ denotes cosine similarity.

$$M_{embed}(u, S_{skills}) = \underset{skill \in S_{skills}}{\operatorname{argmax}} \cos(E(u), E(skill_desc)) \quad (3)$$

Stage 3: LLM Matching

If the first two stages fail, the system invokes the LLM for intent classification:

$$M_{llm}(u, S_{skills}) = \text{LLM}_{\text{intent_classify}}(u, S_{skills}) \quad (4)$$

Full State Transition Based on the above definitions, a complete state transition of EvoAgent is defined as follows:

$$\begin{aligned} h^{(t+1)} &= \text{append}(h^{(t)}, a^{(t)}, result^{(t)}), \\ u^{(t+1)} &= f_u(u^{(t)}, h^{(t+1)}), \\ S_{skills}^{(t+1)} &= f_s(S_{skills}^{(t)}, h^{(t+1)}), \\ c^{(t+1)} &= \text{update_compression}(h^{(t+1)}), \\ s^{(t+1)} &= (h^{(t+1)}, u^{(t+1)}, S_{skills}^{(t+1)}, c^{(t+1)}) \end{aligned} \quad (5)$$

Reward Function In practical EvoAgent applications, there is no explicit numerical reward calculation. Skill tracking primarily relies on implicit monitoring of usage count and success rate. For theoretical formulation, we define the reward function as a weighted combination of skill maturity, user profile update magnitude, and memory update magnitude, assuming equal weights. Here, $\Delta u_{\text{profile}}$ denotes the magnitude of user profile updates, and Δmemory denotes the magnitude of memory updates:

$$R(s, a, s') = w_1 \cdot \text{Maturity}(\text{skill}) + w_2 \cdot \Delta u_{\text{profile}} + w_3 \cdot \Delta \text{memory} \quad (6)$$

Optimization Objective The optimization objective of EvoAgent is to maximize the expected cumulative reward over a time horizon T , meaning that from the user’s perspective, EvoAgent becomes increasingly effective through continued use:

$$\max \mathbb{E} \left[\sum_{t=1}^T \gamma^t \cdot R(s^{(t)}, a^{(t)}, s^{(t+1)}) \right] \quad (7)$$

The expected cumulative reward maximization objective defined in Equation 7 is not computed through explicit numerical reward signals in engineering practice. Instead, it is achieved through a series of **implicit optimization** mechanisms described in Section 3.3. Specifically:

- The improvement of $\text{Maturity}(\text{skill})$ corresponds to frequent and successful invocation of the skill, thereby granting it higher priority in future matching;
- The update magnitudes $\Delta u_{\text{profile}}$ and Δmemory reflect a deeper system understanding of the user, enhancing contextual relevance in subsequent interactions;
- Compiling the MDP optimization objective into an asynchronous offline evolutionary loop is a key design that enables stable self-evolution in the absence of explicit reward signals.

3.3 EvoAgent Framework

EvoAgent operationalizes the formal MDP model described in Section 3.2 into a stable, efficient, and self-evolving system through a carefully designed six-stage execution process. The core idea is to clearly divide system behavior into two coordinated closed loops:

Online Execution Loop: Responsible for responding to user requests in real time and ensuring reliable task execution. This loop prioritizes low latency and high determinism.

Offline Evolution Loop: Runs asynchronously between sessions, learning from historical interactions and optimizing long-term system capabilities. This loop prioritizes deep analysis and systemic optimization.

The coordination of these two loops enables EvoAgent to maintain smooth real-time interaction while continuously enhancing its capabilities. As illustrated in Figure 1, the overall execution process is divided into the following six stages:

(1) Three-Stage Skill Matching (Online Execution Loop) As the entry point of task execution, the system employs a cascaded matching strategy as described in Section 3.2. It first performs low-cost heuristic screening via trigger words; if unsuccessful, it upgrades to embedding-based semantic similarity matching; finally, it resorts to LLM-based intent understanding. This design maximizes retrieval efficiency while ensuring accuracy and robustness.

(2) Skill Injection and Context Assembly (Online Execution Loop) Once a skill is matched, the system performs structured context assembly rather than simply appending text to the prompt. This includes injecting the main instruction file `SKILL.md` and necessary `references/` files on demand, while updating `usage_count` in metadata. This stage ensures the agent is equipped with the required domain knowledge before execution.

(3) Task Execution and Dynamic Compression (Online Execution Loop) The agent executes the task based on the assembled context. To address context window pressure caused by long conversations, the system monitors token usage in real time and triggers a history compression module when approaching the limit. Unlike conventional summarization, EvoAgent prioritizes preserving structured information (e.g., skill references, external links, key data) and forms an “asset index” to maintain reasoning integrity.

(4) Session Termination Detection and Evolution Trigger (Offline Loop Entry) The system supports post-session update processes (configurable as manual or automatic). This context-switching point ensures that computation-intensive evolutionary tasks do not interfere with real-time services.

(5) Asynchronous Evolution Update (Offline Evolution Core) For user profile and memory management, EvoAgent establishes a multidimensional and dynamically updated information maintenance system structured around five mechanisms:

- **Real-time collection mechanism:** Immediately updates the profile based on user-provided business information during dialogue;
- **Initial guidance mechanism:** Proactively collects key dimensions such as main products and target markets during first interaction;
- **Post-session analysis mechanism:** Extracts new information after sessions to update profile and memory;
- **Behavior suggestion mechanism:** Generates profile update suggestions based on behavioral analysis, finalized upon user confirmation;
- **Response guidance mechanism:** Appends 1–2 guiding suggestions after each task to gradually refine user profile information.

This strategy strictly adheres to safety boundary principles, updating profiles only based on explicitly provided user information and avoiding inference of implicit business data from task content.

(6) Skill Maturity Evaluation (Offline Evolution Output) Based on metadata such as `usage_count` and `success_rate`, skills are categorized into four levels: Budding, Growing, Mature, and Proficient. This evaluation provides users with intuitive references of skill reliability and supports quantitative decisions for skill pruning and optimization.

Through this online–offline dual-loop design, EvoAgent organically integrates agent autonomy with system engineering determinism: the online loop ensures reliable interactions, while the offline loop enables long-term sustainable capability growth. Detailed pseudocode of the evolutionary process is provided in Appendix A.

4 Experiments

This section presents experiments centered around three Research Questions (RQs): (1) Does the self-evolution mechanism of EvoAgent have practical value? (2) Is EvoAgent model-agnostic in terms of capability transferability, i.e., how does its overall performance change when the underlying model is replaced? (3) What are the capability boundaries of EvoAgent?

4.1 RQ1

In this experiment, GPT5.2 is used as the base model of EvoAgent. A ReAct-based framework is designed to support complex workflows in foreign trade scenarios, enabling continuous self-evolution. To evaluate the effectiveness of EvoAgent’s evolutionary mechanism, we compare two settings: (i) direct invocation of GPT5.2, and (ii) GPT5.2 integrated with the EvoAgent framework.

4.1.1 Test Case Collection

To ensure the validity and professionalism of the test data in foreign trade scenarios, we interact with a deployed stable version of EvoAgent via scripted procedures, collecting 664 high-quality multi-turn dialogue samples. Each sample contains 8–9 interaction turns, covering typical foreign trade business scenarios and professional queries. From these, 20 samples are randomly selected, resulting in 172 evaluation instances after splitting multi-turn dialogues. This dataset is also used in Section 4.2 to ensure consistency across experiments.

It is important to clarify that the expert-level foreign trade questions in the dataset are not manually selected from successful skill-triggering cases. Instead, user queries are programmatically generated via a parameterized script that exhaustively traverses combinations of product categories, target markets, buyer types, and 12 predefined foreign trade scenarios (e.g., market analysis, quotation response, customs clearance, payment risk, etc.).

Each query is constructed independently of EvoAgent’s runtime skill matching results, and is not conditioned on whether a skill is successfully triggered or executed. Therefore, the resulting dataset represents a structured sampling of the foreign trade problem space rather than a biased selection favoring EvoAgent.

Although these queries are synthetically generated, data collection is conducted within a deployed EvoAgent environment. Therefore, the distribution is consistent with the deployment setting and is not fully independent of the

system. Future work will further evaluate the framework using independently collected foreign trade corpora to test cross-distribution generalization.

4.1.2 EvoAgent as a Capability Amplifier

Capability Enhancement of GPT5.2 via EvoAgent Based on the test samples constructed in Section 4.1.1, we extract professional queries and feed them into (i) standalone GPT5.2 and (ii) GPT5.2 integrated with EvoAgent, respectively. The outputs are then evaluated across multiple dimensions.

Evaluation Design for Model Responses We adopt a multi-level evaluation framework. The evaluation system consists of two layers: basic textual metrics and an LLM-as-Judge scoring mechanism.

At the basic metric level, we use character count statistics, length ratios, and ROUGE-L similarity to quantify textual characteristics. At the core evaluation level, we employ an LLM-as-Judge approach tailored to our scenario [24] (see Table 2), evaluating responses across five dimensions: professionalism, accuracy, completeness, practicality, and language quality. Each dimension is scored on a 1–5 Likert scale, covering terminology correctness, factual accuracy, coverage completeness, actionable feasibility, and linguistic fluency.

To reduce position bias, we randomly shuffle the order of candidate responses during evaluation. Final scores are computed by aggregating the five dimensions, followed by grouping analysis based on intent type and task difficulty.

Table 2: Evaluation Dimensions and Descriptions

Dimension	Label	Description
Professionalism	professionalism	Correctness of domain terminology and logic
Accuracy	accuracy	Factual/policy/data correctness
Completeness	completeness	Coverage of all key aspects of the query
Practicality	practicality	Actionability of the response
Language Quality	language_quality	Fluency, format, and readability

Results Comparison We compare GPT5.2 with and without EvoAgent integration. Results are shown in Table 3.

In the professionalism dimension, EvoAgent-enhanced GPT achieves 4.762, significantly higher than the baseline (2.703), with an improvement of 2.059. In accuracy, EvoAgent achieves 4.238 compared to 2.907, improving by 1.331.

For completeness, the difference is marginal (4.215 vs. 4.331, -0.116), indicating stability. In practicality and language quality, EvoAgent also shows consistent improvements, reaching 4.709 and 4.779 respectively.

Overall, the average score increases from 3.547 (GPT5.2) to 4.541 (EvoAgent + GPT5.2), representing a 27.998% improvement. These results demonstrate that EvoAgent significantly enhances model performance in professionalism, accuracy, practicality, and language quality, while maintaining stability in completeness.

Table 3: Comparison of GPT and EvoAgent-Enhanced GPT Across Five Dimensions

Dimension	GPT	EvoAgent GPT	Difference
professionalism	2.703	4.762	2.059
accuracy	2.907	4.238	1.331
completeness	4.331	4.215	-0.116
practicality	3.744	4.709	0.965
language_quality	4.052	4.779	0.727
Total / Avg	3.547	4.541	27.998%

4.2 RQ2

Model Selection Unlike research-oriented evolutionary agents, EvoAgent is deployed in a real-world foreign trade assistant system. Therefore, in evaluating model transferability, inference cost and deployment feasibility are also key considerations. GPT5.2 relies on API calls, which incur significant cost at scale. To reduce cost while maintaining performance stability, we conduct model substitution experiments.

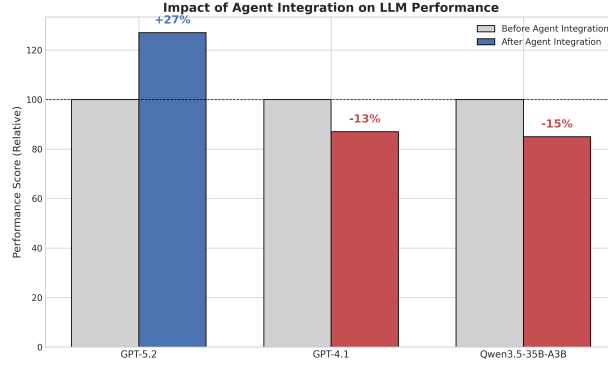


Figure 2: Performance Comparison Before and After EvoAgent Integration. Blue bars represent GPT5.2 performance improvement after integrating EvoAgent; red bars represent GPT4.1 and Qwen performance degradation after integrating EvoAgent; gray bars represent each model’s baseline capability before EvoAgent integration.

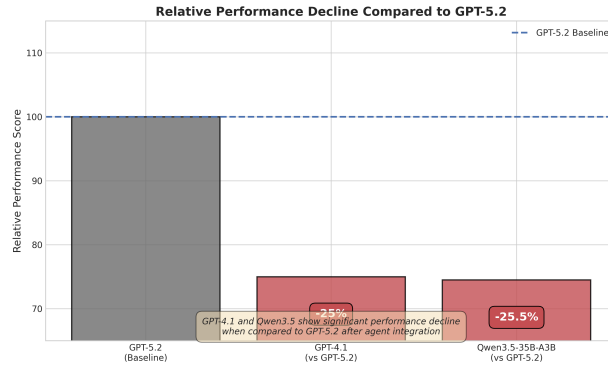


Figure 3: Relative Performance of Models After EvoAgent Integration (vs GPT5.2)

We select GPT4.1 [25] and Qwen3.5-35B-A3B [26] as baselines, representing closed-source API-based and open-source local deployment paradigms respectively.

Results Evaluation metrics are consistent with Section 4.1. We focus on (1) performance change after EvoAgent integration, and (2) performance gap relative to GPT5.2.

GPT4.1 Analysis After integration with EvoAgent, GPT4.1 shows an average performance drop of approximately 13% across five dimensions. Compared to EvoAgent-enhanced GPT5.2, its overall performance is about 75%.

This suggests that EvoAgent does not significantly enhance GPT4.1 under the current configuration, and even introduces performance degradation. This may be due not only to model capability differences, but also to prompt-style mismatch and instruction-following stability issues. Thus, results reflect the interaction between model and system implementation rather than pure architectural gain.

Qwen3.5-35B-A3B Analysis For Qwen, results show that:

- (1) After EvoAgent integration, performance drops to about 85% of its standalone version;
- (2) Compared to EvoAgent-enhanced GPT5.2, Qwen achieves 74.5%;
- (3) Under EvoAgent, Qwen reaches approximately 95% of GPT4.1 performance.

Overall, Qwen shows competitive cost-performance trade-offs among low-cost models (see Figures 2 and 3).

Impact of EvoAgent on Model Capability Overall, EvoAgent exhibits heterogeneous effects across models. For GPT5.2, its structured workflow and multi-stage constraints align well with strong reasoning and tool-use capabilities, leading to performance gains.

In contrast, GPT4.1 and Qwen show performance instability under structured multi-stage constraints. This may be jointly influenced by model capability, prompt compatibility, skill injection format, and context compression strategies. Without ablation studies, we cannot isolate the contribution of each factor.

Thus, we interpret model performance as an interaction between intrinsic reasoning capability and system-level compatibility:

$$Model_capability = \gamma_1 Model_Inference_Capability + \gamma_2 Agent_Implementation_Compatibility \quad (8)$$

4.3 RQ3

4.3.1 Capability Boundary

What EvoAgent Can Do EvoAgent is a foreign trade-oriented intelligent agent system with self-evolution capabilities, comprising three core functional modules.

First, **product reconstruction**. Given competitor links, product images, or related materials, the system can automatically analyze product structure and generate structured output including value propositions, frameworks, and product page copywriting.

Second, **inquiry response generation**. Given customer inquiries, the system generates multiple versions of response emails (e.g., initial reply, quotation confirmation, cold outreach), with controllable tone, length, and strategy.

Third, **foreign trade knowledge QA**. The system provides structured answers to professional foreign trade questions, optionally supported by knowledge base references (in private deployments).

Additionally, EvoAgent supports extensibility via API-based skill management and automatic skill synchronization for new users.

Key System Metrics In performance testing, we evaluate memory capacity and response latency.

For memory, the system supports up to 420 dialogue turns with 6 compression events without runtime errors. Within 120-turn scenarios, no memory errors are observed across 17 tests. Even at 200 turns without compression, full accuracy is maintained. After compression, while some raw details are lost, core semantics are preserved.

Compression is triggered when total context exceeds 64K tokens, balancing memory integrity and system stability. We do not conduct strict benchmark comparisons with standalone GPT5.2 under identical conditions; thus, no direct quantitative comparison is provided.

Overall, EvoAgent demonstrates stable long-horizon operation with only 6 compression events over 420 turns without failures.

For latency, average response time is approximately 12s in early stages, increases to 14s at around 80 turns, and stabilizes at 24s beyond 120 turns. External tool calls add 5–30s latency depending on complexity.

4.3.2 Limitations and Future Work

What EvoAgent Cannot Do Current limitations include:

First, memory constraints: compression leads to irreversible loss of early dialogue details, and long-term memory lacks automatic pruning mechanisms.

Second, user profiling: updates rely on heuristic frequency thresholds and lack deep semantic inference; cross-user knowledge sharing is not supported.

Third, skill execution: only single-skill invocation is supported; no multi-skill orchestration or parallel execution exists.

Fourth, system engineering: lacks cost tracking, monitoring dashboards, and enterprise-grade authentication (e.g., JWT/OAuth).

From a Harness Engineering perspective, the fundamental limitation is the unidirectional coupling between online execution and offline evolution loops. Evolution is triggered only after session termination, preventing real-time adaptation to skill degradation or intent shifts. In other words, EvoAgent supports delayed learning but not real-time adaptation. Addressing this limitation is a key direction for future system evolution.

5 Conclusion

This paper proposes EvoAgent, an evolvable unified agent framework designed for real-world foreign trade scenarios. Through structured skill representation, a user-driven self-evolution mechanism, and a hierarchical task delegation architecture, EvoAgent establishes a closed-loop system integrating skill learning, task execution, and long-term memory management.

From a theoretical perspective, we formalize the evolutionary process as a Markov Decision Process (MDP) and introduce an analytical view in which overall model capability is jointly determined by intrinsic inference ability and agent-model synergy. Empirically, experimental results demonstrate that EvoAgent significantly enhances the performance of high-capability backbone models, while also revealing that the degree of structural coupling between the model and the agent architecture plays a critical role in determining final system effectiveness.

Although the current system still faces limitations in memory compression, multi-skill coordination, and operational support, EvoAgent provides a systematic pathway and practical foundation for building next-generation autonomous agent systems capable of continuous capability growth and real-world deployment.

Acknowledgment

This work was conducted as part of an internal industry project at Focus Technology Co., Ltd. The authors would like to thank the engineering and product teams for their support in system deployment and real-world testing. The views expressed in this paper are solely those of the authors and do not necessarily represent the official position of the company.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [3] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. *arXiv preprint arXiv:2410.02644*, 2024.
- [4] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. Evoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [5] Anthropic. Agent skills overview. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025.
- [6] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [7] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [8] Zhaohan Feng, Ruiqi Xue, Lei Yuan, Yang Yu, Ning Ding, Meiqin Liu, Bingzhao Gao, Jian Sun, Xinhua Zheng, and Gang Wang. Multi-agent embodied ai: Advances and future directions. *Science China Information Sciences*, 69(5):151202, 2026.
- [9] Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D Nguyen. Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322*, 2025.
- [10] Mitchell Hashimoto. Harness engineering, 2026. Concept proposal on harness-based LLM system design.
- [11] OpenAI. Harness engineering: leveraging codex in an agent-first world, February 2026. Accessed: 2026-02-11.
- [12] Birgitta Böckeler and Martin Fowler. Harness engineering, February 2026. Accessed: 2026-02-17.
- [13] Paul Iusztin. Agentic harness engineering: Llms as the new operating system, 2026. Engineering blog post.

- [14] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [15] Tarisai Mugambiwa and Belinda Ndlovu. Multi-agent retrieval augmented generation for clinical decision support: A systematic review and integrative conceptual framework. *Journal of Applied Informatics and Computing*, 10(1):171–183, 2026.
- [16] Yuxiang Zhou, Jichang Li, Yanhao Zhang, Haonan Lu, and Guanbin Li. Mobile-agent-rag: Driving smart multi-agent coordination with contextual knowledge empowerment for long-horizon mobile automation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 29939–29947, 2026.
- [17] Nous Research. Hermes agent. <https://hermes-agent.org/>, 2026.
- [18] Guohao Li et al. Camel: Communicative agents for "mind" exploration of large language model society. *arXiv preprint arXiv:2303.17760*, 2023.
- [19] Qingyun Wu et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [20] Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [21] Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [22] Joon Sung Park et al. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.
- [23] Xingjiao Wu, Luwei Xiao, Yixuan Sun, Junhang Zhang, Tianlong Ma, and Liang He. A survey of human-in-the-loop for machine learning. *Future Generation Computer Systems*, 135:364–381, 2022.
- [24] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, and et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.
- [25] OpenAI. Gpt-4.1 technical report. <https://openai.com/research>, 2025.
- [26] Qwen Team. Qwen3.5 technical report. <https://huggingface.co/Qwen>, 2026.

Appendices

Appendix A: EvoAgent Self-Evolution Process Pseudocode

```

1 Algorithm : EvoAgent Skill Evolution
2 % Note: This pseudocode reflects implementation using OpenAI Agents SDK.
3
4 Input:
5     u           - User input
6     S_skills    - Skills Base
7     u_profile   - USER.md
8     h           - Conversation History
9     $\lambda$   - History Compression Threshold
10    $\theta$    - Embedding Match Threshold (Default 0.6)
11
12 Output:
13     s_out       - Matched Skill
14     h'          - Updated Conversation History
15     u_profile'  - Updated User Profile
16     S_skills'   - Updated Skill Base
17     c'         - Updated Compression Status
18
19 -----
20
21 1: // Initialization
22 2: user_id $\leftarrow$ extract_user_id(u)
23 3: session_id $\leftarrow$ extract_session_id(u)
24 4: ws $\leftarrow$ Workspace(user_id)
25
26 5: // -- Phase 1: Skill Matching (Three Stages) --
27 6: skill $\leftarrow$ $\emptyset$
28 7: // Stage 1: Keyword Matching (O(1) fast screening)

```

```

29 8: for each sk \${in}\$ S_skills do
30 9:   for each t \${in}\$ sk.triggers do
31 10:    if t.lower() \${in}\$ u.lower() then
32 11:     skill \${leftarrow}\$ sk
33 12:     match_type \${leftarrow}\$ "keyword"
34 13:     confidence \${leftarrow}\$ 1.0
35 14:     goto PHASE2
36 15:   end if
37 16: end for
38 17: end for
39
40 18: // Stage 2: Embedding Matching (cosine similarity)
41 19: if skill = \${emptyset}\$ then
42 20:   e_u \${leftarrow}\$ get_embedding(u)           // Call embedding API
43 21:   best_score \${leftarrow}\$ 0
44 22:   for each sk \${in}\$ S_skills do
45 23:     e_sk \${leftarrow}\$ get_skill_embedding(sk) // Cached embedding
46 24:     score \${leftarrow}\$ cosine_similarity(e_u, e_sk)
47 25:     if score > best_score then
48 26:       best_score \${leftarrow}\$ score
49 27:       skill \${leftarrow}\$ sk
50 28:     end if
51 29:   end for
52 30:   if best_score \${geq}\$ \${theta}\$ then
53 31:     match_type \${leftarrow}\$ "embedding"
54 32:     confidence \${leftarrow}\$ best_score
55 33:   else
56 34:     skill \${leftarrow}\$ \${emptyset}\$
57 35:   end if
58 36: end if
59
60 37: // Stage 3: LLM Matching (semantic fallback)
61 38: if skill = \${emptyset}\$ then
62 39:   skill \${leftarrow}\$ LLM_intent_classify(u, S_skills)
63 40:   if skill \${neq}\$ \${emptyset}\$ then
64 41:     match_type \${leftarrow}\$ "llm"
65 42:     confidence \${leftarrow}\$ 0.7
66 43:   end if
67 44: end if
68
69 45: PHASE2:
70 46: // -- Phase 2: Skill Injection & Context Assembly --
71 47: if skill \${neq}\$ \${emptyset}\$ then
72 48:   // Context Engineering: inject skill via synthetic tool call
73 49:   // (OpenAI Agents SDK: either embedded in instructions or appended as tool
    messages)
74 50:   h \${leftarrow}\$ h \${oplus}\$ {role: "assistant", tool_calls: [{
75 51:     id: "skill_load",
76 52:     function: {name: "skill_loader", arguments: {skill: skill.name}}
77 53:   ]}]
78 54:   h \${leftarrow}\$ h \${oplus}\$ {role: "tool", tool_call_id: "skill_load",
79 55:     content: format_skill_content(skill)}
80
81 56:   // Increment usage tracking metadata
82 57:   skill.usage_count \${leftarrow}\$ skill.usage_count + 1
83 58:   skill.updated_at \${leftarrow}\$ now()
84 59:   ws.skill_set(skill.name, skill_to_md(skill))
85 60: end if
86
87 61: // -- Phase 2.5: Optional Sub-Agent Handoff (if required by skill) --
88 62: if skill \${neq}\$ \${emptyset}\$ and skill.requires_sub_agent then
89 63:   sub_agent \${leftarrow}\$ Agent(name=skill.sub_agent_name,
90 64:     instructions=skill.sub_agent_instructions,
91 65:     tools=skill.sub_agent_tools)

```

```

92 66:     result \$\\leftarrow\$ Runner.run(sub_agent, input=h)    // Sub-agent
      execution
93 67:     h \$\\leftarrow\$ h \$\\oplus\$ result.to_input_list()
94 68:     goto PHASE4    // Sub-agent result flows back; bypass main agent execution
95 69: end if
96
97 70: // -- Phase 3: Task Execution (Main Agent) --
98 71: // Build dynamic instructions incorporating skill content
99 72: instructions \$\\leftarrow\$ build_instructions(SOUL.md, USER.md, MEMORY.md,
      skill)
100 73: main_agent \$\\leftarrow\$ Agent(name="EvoAgent",
101 74:         instructions=instructions,
102 75:         tools=tools)
103 76: result \$\\leftarrow\$ Runner.run_streamed(main_agent, input=h)
104 77: h \$\\leftarrow\$ h \$\\oplus\$ result.to_input_list()
105
106 78: // -- Phase 4: Session End Detection & Offline Evolution --
107 79: if session_ended then
108 80:     // Independent review agent (asynchronous Sub-Agent)
109 81:     review_agent \$\\leftarrow\$ Agent(
110 82:         name="Session Reviewer",
111 83:         instructions="You are an offline analyst extracting profile/memory
      changes and reusable skills.",
112 84:         tools=[UpdateUserProfileTool, UpdateMemoryTool, ExtractSkillTool]
113 85:     )
114 86:     review_result \$\\leftarrow\$ await Runner.run(review_agent, input=h)
115
116 87:     // Extract and apply profile updates (conservative update principle)
117 88:     \$\\Delta\$u_profile \$\\leftarrow\$ LLM_extract_profile_changes(h)
118 89:     u_profile \$\\leftarrow\$ u_profile \$\\oplus\$ \$\\Delta\$u_profile
119 90:     ws.write_prompt("USER", u_profile)
120
121 91:     // Extract and apply memory updates
122 92:     \$\\Delta\$memory \$\\leftarrow\$ LLM_extract_memory_changes(h)
123 93:     memory \$\\leftarrow\$ memory \$\\oplus\$ \$\\Delta\$memory
124 94:     ws.write_prompt("MEMORY", memory)
125
126 95:     // Extract new skills (with quality gating)
127 96:     new_skills \$\\leftarrow\$ LLM_extract_skills(h)
128 97:     for each sk \$\\in\$ new_skills do
129 98:         ws.skill_set(sk.name, skill_to_md(sk))
130 99:     end for
131 100: end if
132
133 101: // -- Phase 5: History Compression (asset-preserving summarization) --
134 102: if should_compress(h) then
135 103:     // 1. Extract asset index (skills, references, images, URLs)
136 104:     assets \$\\leftarrow\$ extract_asset_index(h)
137 105:     // 2. Generate structured 9-part summary (Claude Code style)
138 106:     summary \$\\leftarrow\$ llm_generate_structured_summary(h)
139 107:     // 3. Retain recent N messages, replace older with summary + assets
140 108:     h_compressed \$\\leftarrow\$ compress_tail_messages(h)
141 109:     h \$\\leftarrow\$ {summary: summary, assets: assets} \$\\oplus\$
      h_compressed
142 110: end if
143
144 111: // -- Phase 6: Skill Maturity Assessment --
145 112: if skill \$\\neq\$ \$\\emptyset\$ then
146 113:     u \$\\leftarrow\$ skill.usage_count
147 114:     sr \$\\leftarrow\$ skill.success_rate
148 115:     if u \$\\geq\$ 10 and sr \$\\geq\$ 0.85 then
149 116:         maturity \$\\leftarrow\$ "Proficient"
150 117:     else if u \$\\geq\$ 4 and sr \$\\geq\$ 0.7 then
151 118:         maturity \$\\leftarrow\$ "Mature"
152 119:     else if u \$\\geq\$ 1 then

```

EvoAgent

```
153 120:      maturity \\\leftarrow$ "Growing"
154 121:      else
155 122:      maturity \\\leftarrow$ "Budding"
156 123:      end if
157 124: end if
158
159 125: return skill, h, u_profile, S_skills, c
```

Appendix B: EvoAgent Git

EvoAgent:<https://github.com/Focus-AI-Center/Mentarc-EvoAgent>